BD1_18-02-23

AUTHOR (Name, Nationality)
PUBLISHER (Name, City, Nation)
BOOK (Title, Publisher, PageNumber, Language, Price, Currency, Year)
AUTHOR_RIGHTS (Author, Book, Publisher, Percentage)

1. Define the schema of AUTHOR_RIGHTS, including the referential integrity constraints.

```
CREATE TABLE AUTHOR_RIGHTS (
Author VARCHAR(255) REFERENCES AUTHOR (Name)
ON UPDATE CASCADE ON DELETE NO ACTION
Book VARCHAR(255)
Publisher VARCHAR(255)
Percentage FLOAT NOT NULL
FOREIGN KEY (Book, Publisher) REFERENCES BOOK (Title, Publisher)
ON UPDATE CASCADE ON DELETE NO ACTION
PRIMARY KEY(Author, Book, Publisher)
)
```

2. For each publisher, find the authors who have published books with that publisher in at least 10 different languages. The result should list the publishers in alphabetical order, and for each publisher, the authors should be sorted by the number of different languages (from highest to lowest).

```
SELECT B.Publisher, Author
FROM BOOK B, AUTHOR_RIGHTS A
WHERE Title = Book AND B.Publisher = A.Publisher
GROUP BY B.Publisher, Author
HAVING COUNT(DISTINCT Language) >= 10
ORDER BY B. Publisher ASC, COUNT(DISTINCT Language) DESC
```

3. Find authors who have written books for every publisher in Milan without ever receiving more than a 5% commission from them.

```
SELECT A.Author
FROM AUTHOR_RIGHTS A, PUBLISHER P
WHERE A.Publisher = P.Name AND P.City = "Milan"
GROUP BY A.Author
HAVING MAX(A.Percentage) <= 5 AND COUNT(DISTINCT P.Name) = (
                                                    SELECT COUNT(*)
                                                    FROM PUBLISHER
                                                    WHERE City = "Milano")
```

Alternatively

```
SELECT D.Author
FROM AUTHOR_RIGHTS D, PUBLISHER P
WHERE D. Publisher = P.Name AND P.City = "Milano"
AND D. Author NOT IN (SELECT D1. Author
                      FROM AUTHOR_RIGHTS D1, PUBLISHER P1
                      WHERE D1. Publisher = P1.Name AND P1.City = "Milano" AND
                      D1.Percentage > 5)
GROUP BY D.Author
HAVING COUNT(DISTINCT P.Name) = (SELECT COUNT(*)
                                 FROM Publisher
                                 WHERE City = "Milano")
```

4. Express in optimized relational algebra and in Datalog the following query:
Find Italian authors who wrote only books in Italian before 2015 and only books in other languages from 2015 onward.

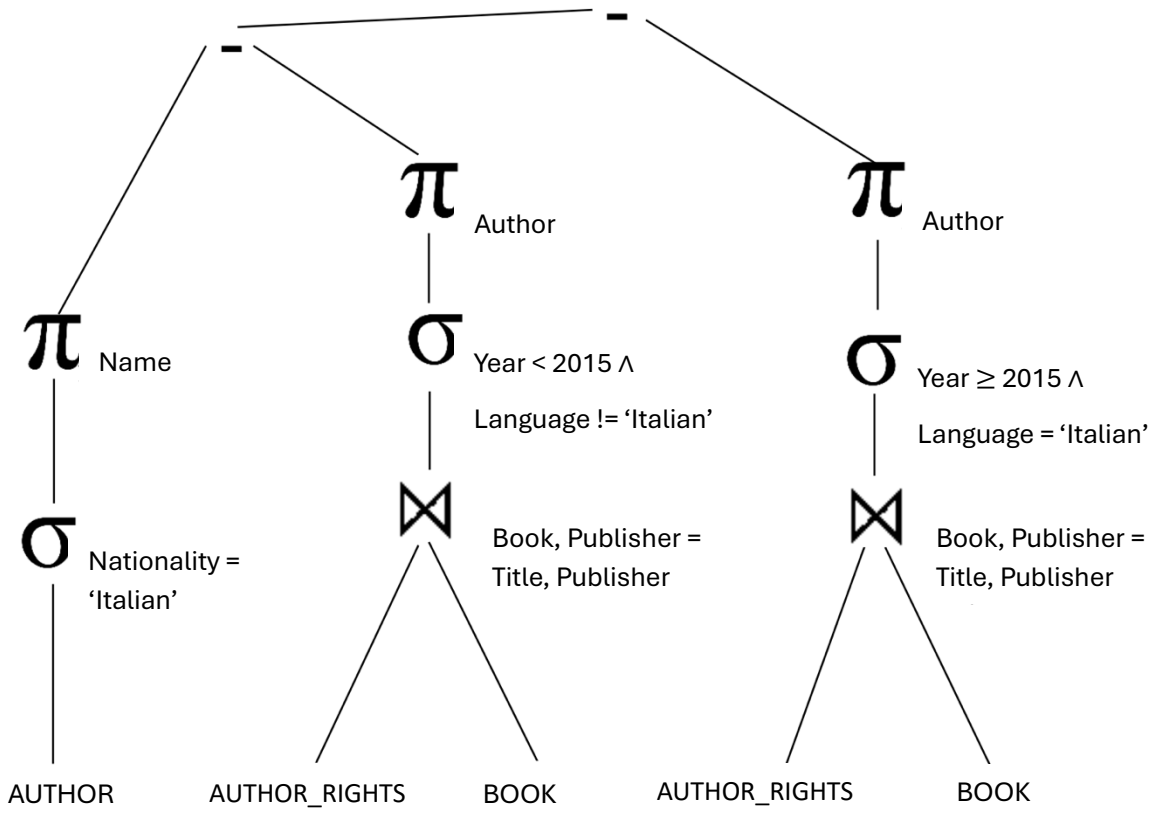
DATALOG

DIFFERENT_ITA_BEFORE_2015(A) :- AUTHOR_RIGHTS (A, B, P, _), BOOK (B, P, _, Lan, _, _, Year), Year < 2015, Lan != 'Italian'

ITA_FROM_2015(A) :- AUTHOR_RIGHTS(A, B, P, _), BOOK(B, P, _, 'Italian', _, _, Year), Year ≥ 2015

SOL(A) :- AUTHOR(A, Nat), ¬ DIFFERENT_ITA_BEFORE_2015(A), ¬ ITA_FROM_2015(A), Nat = 'Italian'

?- SOL (x)



TdE_BD1_2024_06-06

The following DB schema describes the Formula 1 car races. Note that ChequeredFlag is True if the pilot completes the grand prix, and that TyreType includes various predefined types (Red, Yellow, White, ...).

PILOT (PilotName, TeamName)
GRAND_PRIX (GrandPrixName, Country, Continent)
RANKINGS (GrandPrixName, PilotName, QualifyingPosition, RacePosition, Points, ChequeredFlag, TotalNrOfLaps)
USED_TYRES (GrandPrixName, PilotName, TyreType, LapsCount)

1. Define the schema of the RANKINGS table, including referential integrity constraints and attribute nullability. Express the constraint that pilots who did not finish the race must always be assigned 0 points.

```
CREATE TABLE RANKINGS (
GrandPrixName VARCHAR(32) REFERENCES GRAND_PRIX(GrandPrixName)
ON UPDATE CASCADE ON DELETE NO ACTION,
PilotName VARCHAR(32) REFERENCES PILOT(PilotName)
ON UPDATE CASCADE ON DELETE NO ACTION,
QualifyingPosition INT NOT NULL CHECK(QualifyingPosition > 0),
RacePosition INT NOT NULL CHECK(RacePosition > 0),
Points INT NOT NULL CHECK(Points >= 0),
ChequeredFlag BOOL NOT NULL CHECK(ChequeredFlag = False AND Points = 0 OR
ChequeredFlag = TRUE),
TotalNrOfLaps INT NOT NULL CHECK(TotalNrOfLaps >=0),
PRIMARY KEY(GrandPrixName, PilotName))
```

2. Extract the pilots who finished a Grand Prix using three different types of tyres.

```
SELECT DISTINCT PilotName
FROM RANKINGS NATURAL JOIN USED_TYRES
WHERE ChequeredFlag = TRUE
GROUP BY PilotName, GrandPrixName
HAVING COUNT(DISTINCT TyreType) = 3; (or >=3)
```

3. Extract the pilots who finished in the top three in all the Grand Prix held in Europe.

```
SELECT PilotName
FROM RANKINGS NATURAL JOIN GRAND PRIX
WHERE ChequeredFlag = TRUE AND Continent = "Europe" AND RacePosition <= 3
GROUP BY PilotName
HAVING COUNT(*) = (
                        SELECT COUNT(*)
                        FROM GRAND PRIX
                        WHERE Continent = "Europe")
```

4. Build the Constructors' Standings: write a view that returns the name of the team followed by the sum of the points obtained by its pilots in descending order of points. Then find the difference in points between the first and the last team in the ranking.

```
CREATE VIEW ConstructorStandings(TeamName, TotalPoints) AS
SELECT TeamName, SUM(Points)
FROM Pilot NATURAL JOIN Rankings
GROUP BY TeamName
ORDER BY SUM(Points) DESC;

SELECT MAX(TotalPoints) - MIN(TotalPoints)
FROM ConstructorStandings
```

5. Express in Relational Algebra and Datalog the following query:
Extract the pilots who have never finished in the top ten in European Grands Prix but arrived at least once in the top three in qualifying (considering only European Grands Prix).

DATALOG

```
Pilots10EU(P) :- Grand_Prix(GP, _ "Europe"), Rankings(GP, Q, _, R, _, _, _), R <= 10
PilotsQual3EU(P) :- Grand_Prix(GP, _, "Europe"), Rankings(GP, P, Q, _, _, _, _), Q <= 3
PilotsOK(P) :- PilotsQual3EU(P), ¬ Pilots10EU(P)
?-PilotsOK(x)
```

RELATIONAL ALGEBRA

